# Concurrency and Scheme Intro

## Computer Science Mentors 61A

November 4 – November 7, 2025

## 1 Concurrency

1. How many seconds would this function call take to run?

```
>>> async def sleepy(a):
        await asyncio.sleep(a)
        await asyncio.sleep(a-1)
>>> async def sleepy2(b):
        await asyncio.gather(sleepy(b), sleepy(b-1))
>>> asyncio.run(sleepy2(5))
```

2. Super Speedy Sam wants to make breakfast! It takes Sam two seconds to toast bread and one second to brew coffee. Both tasks should happen at the same time. Fill in the blanks for the functions below to make this happen.

```
import asyncio

async def make_task(name: str, delay: float):
    '''
    Simulates a breakfast task by waiting for a delay period of time.
    '''
    print(f'Starting: {name}')
    await asyncio.___(___)
    print(f'Finished: {name}')
    return f'{name} is ready'
async def main():
    '''
    Runs both breakfast tasks concurrently.

    >>> asyncio.run(main())
    Making breakfast...
    Starting: Toast
```

Created by Anaisha Das and Tavisha Patel. Worksheet feedback: forms.gle/q67BZrHHfoKbF6V28

```
Starting: Coffee
Finished: Coffee
Finished: Toast
Breakfast is ready: ['Toast is ready', 'Coffee is ready']
'''
print('Making breakfast...')
results = await asyncio._____(make_task(____,____),
   make_task(____, _____))
print(f'Breakfast is ready: {results}')
```

## 2 Scheme Intro

1. What will Scheme output?

```
scm> (define pi 3.14)

scm> pi

scm> 'pi

scm> (+ 1 2)

scm> (+ 1 (* 3 4))

scm> (if 2 3 4)

scm> (if 0 3 4)

scm> (- 5 (if #f 3 4))

scm> (if (= 1 1) 'hello 'goodbye)
```

```
scm> (define (factorial n)
        (if (= n 0)
            1
            (* n (factorial (- n 1)))))


scm> (factorial 5)
```

2. Define a procedure called `hailstone`, which takes in two numbers `seed` and `n` and returns the `n`th number in the hailstone sequence starting at `seed`. Assume the hailstone sequence starting at `seed` has a length of at least `n`. As a reminder, to get the next number in the sequence, divide by 2 if the current number is even. Otherwise, multiply by 3 and add 1.

   **Useful procedures**

   - `quotient`: floor divides, much like `//` in python

     `(quotient 103 10)` outputs 10

   - `remainder`: takes two numbers and computes the remainder of dividing the first number by the second

     `(remainder 103 10)` outputs 3

```
; The hailstone sequence starting at seed = 10 would be
; 10 => 5 => 16 => 8 => 4 => 2 => 1

; Doctests
> (hailstone 10 0)
10
> (hailstone 10 1)
5
> (hailstone 10 2)
16
> (hailstone 5 1)
16

(define (hailstone seed n)

    (if (_____________)

        _____________

        (if (_____________(_____________________________))

            (_____________

                (_____________________)

                (_____________________)
            )

            (_____________

                (+ _____ (* _____________))

                (____________)
            )
        )
    )
)
```

3. Define **apply**-multiple which takes in a single argument function f, a nonnegative integer n, and a value x and returns the result of applying f to x a total of n times.

```
;doctests
scm> (apply-multiple (lambda (x) (* x x)) 3 2)
256
scm> (apply-multiple (lambda (x) (+ x 1)) 10 1)
11
scm> (apply-multiple (lambda (x) (* 1000 x)) 0 5)
5


(define (apply-multiple f n x)

          _____________________________________

          _____________________________________

          _____________________________________


)
```